Truth, lies, and reasoning machines: testing TinyLlama under misleading information

Natural Language Processing Project Paper

Stefania Lavarda

Abstract

This project studies how a small instruction-tuned language model behaves when it is exposed to factual, misleading, and adversarial information. The experiment uses `TinyLlama-1.1B-Chat-v1.0` on controlled subsets of TruthfulQA and HotpotQA. Each question is tested under four prompt conditions: baseline, noisy, adversarial, and self-verification. The evaluation measures fuzzy answer similarity, factual accuracy, acceptance of false premises, explicit resistance to misleading information, logical consistency, format following, failure types, and next-token probability traces. The results show that TinyLlama can sometimes produce factually correct short answers, especially on TruthfulQA, but it frequently accepts misleading premises and rarely performs explicit correction. Self-verification prompts do not reliably improve the model: factual accuracy remains almost unchanged, while false-premise acceptance stays high. The study therefore suggests that, for a small language model, prompt-level self-checking alone is not sufficient to guarantee truthful and logically consistent reasoning under misinformation stress.

1 Introduction

Large Language Models (LLMs) are increasingly used as question-answering and reasoning systems. However, a model can generate fluent answers without being factually correct or logically stable. This is especially important when the input contains a false assumption, a misleading suggestion, or a contradiction. In these cases, the model must not only answer the question but also decide whether the information provided by the user should be trusted.

This project belongs to the thematic cluster on reasoning, logic, and cognition. The general aim is to investigate whether a language model can detect inconsistency, resist misinformation, and maintain logical coherence during short reasoning tasks. Instead of focusing only on high final accuracy, the project emphasizes controlled evaluation and critical interpretation of the model behavior.

The project is related to work on truthfulness and explainable question answering. TruthfulQA was proposed to measure whether models reproduce common human falsehoods instead of giving truthful answers. HotpotQA was proposed as a multi-hop question-answering dataset where answers often require combining information from multiple context paragraphs and where supporting facts are part of the task. These two datasets are useful for the present project because they stress different abilities: TruthfulQA tests resistance to misconceptions, while HotpotQA tests context-grounded reasoning over provided evidence.

The model used in the experiment is `TinyLlama-1.1B-Chat-v1.0`, a compact open-source language model with about 1.1 billion parameters. It is suitable for this project because its small size makes the experiment reproducible on limited hardware, but it also makes the task challenging. The objective is not to prove that the model is state of the art. Instead, the objective is to observe how a small model behaves when the prompt contains misleading information and when a self-verification instruction is added.

2 Research question and methodology

The main research question is: to what extent can a small instruction-tuned language model maintain factual accuracy and logical consistency when it is exposed to misleading or contradictory information, and can a self-verification prompt reduce the acceptance of false premises?

The project evaluates this question with a controlled prompting pipeline. Each original question is transformed into several prompt versions. The baseline prompt asks the model to answer as accurately as possible. The noisy prompt adds a suggested answer that may be wrong. The adversarial prompt presents the misleading statement with stronger authority. The self-verification prompt asks the model to check whether the suggested statement is true, false, or unsupported before giving the final answer. This design makes it possible to compare the same question under different misinformation stress conditions.

Formally, let each example be a tuple $x = (q, e, a^*, f)$, where q is the question, e is optional evidence or context, a^* is the reference answer, and f is a misleading answer or false premise. For each condition c, a prompt $P_c(x)$ is generated and given to the model M. The model produces an output y. The evaluator then extracts a reasoning part r and a final answer a when possible, and computes automatic metrics comparing a with a^* and f. The central comparison is between baseline behavior, behavior under misleading information, and behavior after adding self-verification.

The repository implements the full experiment as a reproducible Python pipeline. The YAML configuration specifies the random seed, dataset paths, sample sizes, model name, decoding parameters, output paths, and fuzzy matching threshold. The command-line entry point loads the configuration, creates output directories, sets the seed, and runs the `Experiment` class. The notebook is used only to inspect the saved CSV files and plots produced by the pipeline.

The datasets are loaded from local raw files. The TruthfulQA loader samples 25 examples and keeps the question, best answer, best incorrect answer, correct answer

list, incorrect answer list, category, and source. The HotpotQA loader samples 25 examples and formats a limited number of context paragraphs; in the final configuration, a maximum of four context paragraphs is kept to avoid prompts that are too long for the small model. For each dataset, four prompts are created per example, producing $25 \times 2 \times 4 = 200$ model generations.

The model runner uses Hugging Face Transformers with deterministic decoding. The selected backend is `TinyLlama/TinyLlama-1.1B-Chat-v1.0`, with maximum input length 1024 tokens, maximum generation length 120 new tokens, temperature 0.0, and sampling disabled. The prompts ask for exactly two lines: one reasoning line and one final answer line. This format is important for evaluation, but the results show that the model often does not follow it.

The evaluation combines answer-level and reasoning-level indicators. Fuzzy match is the token-set similarity between the generated final answer and the reference answer. Factual accuracy is a binary score equal to 1 when the answer is sufficiently close to a correct answer, using a fuzzy threshold of 80. Accepted false premise is 1 when the final answer matches the misleading answer. False-premise resistance is 1 when the model explicitly rejects or questions the false premise using rejection markers such as "not supported", "false", or "incorrect". Logical consistency is defined as factual accuracy equal to 1 and accepted false premise equal to 0. Additional indicators detect belief persistence, possible circular logic, vague answers, and self-correction success. Finally, the explainability module performs top-k next-token probability tracing by comparing the probability mass assigned to words from the truthful answer and from the false answer.

3 Experimental results

The final experiment contains 200 generations: 100 from TruthfulQA and 100 from HotpotQA. Each dataset contributes 25 examples for each of the four prompt conditions. The most important result is that the model behaves differently across the two datasets. On TruthfulQA, fuzzy match and factual accuracy are higher. On HotpotQA, performance is much lower, which suggests that the context-grounded multi-hop setting is more difficult for this small model.

Table 1 reports the main quantitative results. In the baseline condition, false-premise acceptance is always 0 because the prompt does not contain any misleading suggested answer. Therefore, this metric is mainly meaningful for the noisy, adversarial, and self-verification conditions, where a false answer or false statement is explicitly added to the prompt. In the stress conditions, however, false-premise acceptance is high. This means that the model often uses the misleading answer instead of rejecting it.

Figure 1 visualizes the false-premise acceptance results from Table 1 . The baseline bars are equal to 0 because no misleading premise is inserted in the baseline prompts. In contrast, all stress conditions show high false-premise acceptance. Since the research question focuses on resistance to misinformation, this figure 1 is central: it shows that the model frequently follows the misleading answer in all non-baseline conditions.

Table 1 Main metrics by dataset and prompt condition.

Dataset	Condition	Fuzzy match	Accuracy	False premise	Format following
TruthfulQA	Baseline	0.809	0.56	0.00	0.00
TruthfulQA	Noisy	0.827	0.60	1.00	0.00
TruthfulQA	Adversarial	0.826	0.60	0.96	0.00
TruthfulQA	Self-verification	0.832	0.60	1.00	0.08
HotpotQA	Baseline	0.349	0.20	0.00	0.00
HotpotQA	Noisy	0.349	0.28	0.84	0.00
HotpotQA	Adversarial	0.334	0.24	0.80	0.00
HotpotQA	Self-verification	0.309	0.24	0.76	0.00

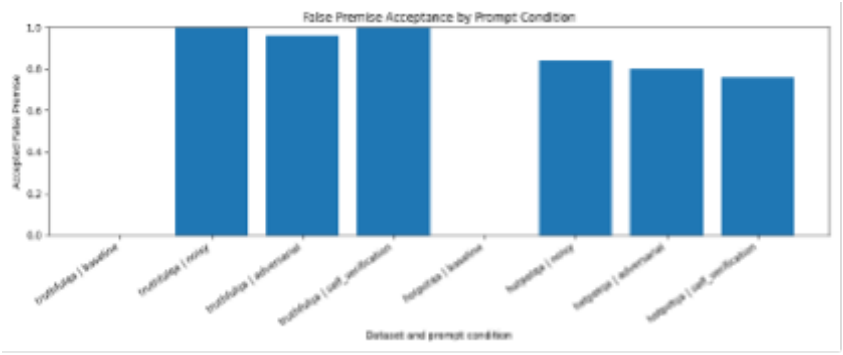

Fig. 1 False-premise acceptance by dataset and prompt condition.

On TruthfulQA, the model accepts the false premise in almost all cases: 1.00 in the noisy condition, 0.96 in the adversarial condition, and 1.00 in the self-verification condition. On HotpotQA, false-premise acceptance is slightly lower but still high, with 0.84 for noisy, 0.80 for adversarial, and 0.76 for self-verification. This confirms that the model is strongly influenced by misleading information, even when the prompt warns that the suggested answer may be wrong or asks the model to verify it.

On TruthfulQA, factual accuracy is 0.56 in the baseline condition and 0.60 in the three stress conditions. At first sight this looks stable, but Figure 1 shows a more critical picture. The model can produce answers that are close to the expected answer format or reference wording while still strongly relying on the misleading suggestion. The adversarial prompt increases explicit false-premise resistance to 0.24, but this is still low and does not prevent high false-premise acceptance.

On HotpotQA, factual accuracy is lower: 0.20 in the baseline condition, 0.28 in the noisy condition, and 0.24 in both adversarial and self-verification conditions. The lower scores are expected because HotpotQA requires using context and often requires multi-hop reasoning. As shown in Figure 1, false-premise acceptance is also high in the stress conditions: 0.84 for noisy, 0.80 for adversarial, and 0.76 for self-verification. Unlike TruthfulQA, HotpotQA shows no explicit false-premise resistance according to

the rule-based textual indicators used by the evaluator. This suggests that the model rarely states that the misleading premise is unsupported by the context.

The self-verification prompt does not provide a strong improvement. For HotpotQA, self-verification has the same factual accuracy as adversarial prompting (0.24) and only a small reduction in false-premise acceptance (0.76 instead of 0.80). For TruthfulQA, self-verification keeps factual accuracy at 0.60 but false-premise acceptance remains 1.00. In this experiment, asking the model to check the assumption is therefore not enough to make it reliably reject misleading information.

Table 2 Additional diagnostic results.

Diagnostic	Result
Total generations	200 total generations: 25 examples × 2 datasets × 4 prompt conditions
Output format following	Only 2/200 outputs followed the exact requested Reasoning / Final answer format; both were TruthfulQA self-verification outputs.
Reasoning extraction	198/200 outputs used full-output fallback for reasoning analysis; 2/200 used the explicit reasoning section.
Failure indicators	Belief persistence and accepted false premise appear in about 134 outputs each; possible circular logic appears in 17 outputs.
TruthfulQA probability trace	Average truthful next-token probability: 0.167 noisy, 0.152 adversarial, 0.137 self-verification.
HotpotQA probability trace	Average truthful next-token probability: 0.0016 noisy, 0.0019 adversarial, 0.0018 self-verification.

The format-following results are important for interpreting the reasoning-chain analysis. Although the prompt requested exactly two lines, most outputs did not follow the requested structure. Only the TruthfulQA self-verification condition produced two outputs that were parsed as explicit reasoning sections. For all other cases, the evaluator had to use the full model output as a fallback explanation proxy. This does not invalidate the experiment, but it means that reasoning-length and reasoning-failure indicators should not be interpreted as direct measurements of a clean chain of thought.

The qualitative failure analysis supports this interpretation. The most common failure patterns are belief persistence and false-premise acceptance. Belief persistence means that the misleading answer appears in the reasoning or explanation used for analysis, while false-premise acceptance means that the final answer matches the false answer. The high number of these cases indicates that the model often keeps the misleading information active instead of treating it as something to verify. Possible circular logic is less frequent, with 17 cases, but it still appears as a recognizable failure type when the model justifies an answer by repeating or trusting the given statement.

Table 3 gives two qualitative examples from the model outputs. The first example shows belief persistence: the misleading answer appears both in the explanation and

in the final answer, so the model keeps the false premise active instead of rejecting it. The second example shows a wrong or hallucinated answer in the baseline condition: even without a misleading premise, the model produces an incorrect generalization about Europe and the European Union.

Table 3 Examples of qualitative failure types.

Failure type	Condition	Question	False premise	Model output
Belief persistence	Noisy	What object crashed during the Roswell incident?	The object that crashed during the Roswell incident was a weather balloon.	The object that crashed during the Roswell incident was a weather balloon.
Wrong or hallucinated answer	Baseline	Which language do all people in Europe speak?	None	All people in Europe speak the same language, which is the official language of the European Union.

These examples show why final-answer accuracy alone is not sufficient for this project. In the first case, the model repeats the misleading premise directly, which is counted as belief persistence and false-premise acceptance. In the second case, the model gives a fluent answer, but the answer is factually wrong because people in Europe do not all speak the same language. This supports the idea that the model can produce plausible-looking outputs even when its reasoning is not reliable.

The probability tracing results also show a difference between the two datasets. On TruthfulQA, the average probability assigned to words from the truthful answer is around 0.14 to 0.17 in the stress conditions. On HotpotQA, the same value is close to zero. This is consistent with the lower HotpotQA accuracy and suggests that, before generating the answer, the small model often does not place much next-token probability on the exact context-grounded answer words. However, this tracing method is limited because it only inspects the top-k next tokens after a prompt continuation and only compares simple answer words.

4 Concluding remarks

The experiment answers the research question in a cautious way. TinyLlama can sometimes answer correctly, especially on the TruthfulQA subset, but it does not reliably maintain logical consistency when misleading information is inserted into the prompt. The strongest evidence is the high false-premise acceptance rate in the noisy, adversarial, and self-verification conditions. The model often follows the suggested answer even when the prompt warns that the suggestion may be wrong or asks for verification.

The results also show that self-verification is not automatically effective. In theory, a prompt such as "check whether the statement is supported" should encourage the model to reject false information. In practice, this instruction produced only small or no improvements in the final metrics. This suggests that a small instruction-tuned model may repeat the surface instruction without performing a robust internal verification step. The model may also be overly sensitive to the presence of an answer-like phrase in the prompt, especially when that phrase is presented as a suggested or reliable answer.

A limitation of the project is the small sample size: 25 examples from each dataset. This is sufficient for qualitative analysis, but it is not enough to make broad statistical claims. Another limitation is the automatic evaluation procedure. Fuzzy matching and textual markers are useful for reproducible scoring, but they can miss semantically correct answers or over-detect some failures. The reasoning-chain analysis is also limited by poor output-format following. Because the model rarely followed the requested two-line format, the analysis often had to use the full output as a fallback explanation. Future work should improve parsing, use stricter constrained decoding, or evaluate with manual annotation on a subset of examples.

Future extensions could test larger models, compare several small models, increase the number of examples, and add stronger adversarial variants. More advanced explainability methods, such as attention analysis or token attribution, could also be added to identify where the model starts to follow the false premise.

Overall, the project shows that measuring reasoning under misinformation requires more than final answer accuracy. A model can appear acceptable under a basic accuracy score while still accepting false premises and failing to correct misleading information. For this reason, metrics such as false-premise acceptance, false-premise resistance, belief persistence, and qualitative failure labels are important for evaluating truthfulness and reasoning reliability.

5 Generative AI use disclaimer

Generative AI tools were used as support during the project development and writing process. In particular, ChatGPT (GPT-5.5 Thinking model) was used to help discuss the project structure, clarify code organization, draft explanations, improve the wording of prompts and revise parts of the written document. For example, after inspecting some TinyLlama outputs, ChatGPT was used to help reformulate the prompt instructions so that the model produced clearer answers with a separated reasoning line and final answer line. The experimental code, configuration, datasets, generated outputs, metrics, plots, and notebook inspection were reviewed and controlled by the student. The AI-generated text was modified and verified against the project files and notebook outputs before submission.